

Functional Programming in C++

from a Scala perspective

Massimiliano Pagani
maxpag@gmail.com
www.maxpagani.org

September 16, 2026
Italian C++ Community Meetup

λ

$f: A \rightarrow B$ $g: B \rightarrow C$

$g \circ f: A \rightarrow C$

$id_A: A \rightarrow A$

$x \rightarrow f \rightarrow g \rightarrow h \rightarrow \dots \rightarrow z$

```
#include <functional>

template <typename F, typename G>
auto compose(F f, G g) {
    return [=](auto&& x) {
        return g(f(std::forward<decltype(x)>(x)));
    };
}

auto inc = [](int x) { return x + 1; };
auto square = [](int x) { return x * x; };

auto f = compose(square, inc);
// f(x) = (x + 1)2
```

C++

Who's This Guy?

A grumpy old programmer that worked in C++ for 30+ years

Exposed to (BASIC) C, Java, PHP, C#, bash, 😊 Scala

Main focus embedded system.

Worked in Video Games, Multimedia, Home/Building Automation, Data Collection, now Flight Software.



Agenda

- What is Functional Programming?
- C++ vs Scala — the touchstone
- The three pillars: immutability, pure functions, first-class functions
- Higher-order functions, functors, monads
- I/O and side effects
- Conclusions and takeaways

The Basics

What is functional programming?

What is Functional Programming?

C++ programmer knows functions...

```
void doSomething() {  
    x = x + 1;  
}
```

C++

What is Functional Programming?

Not this kind of function:

```
void doSomething() {  
    x = x + 1;  
}
```

C++

But this kind of function:

$$y = f(x)$$

Same input → always same output. No hidden state.

A Brief History

Alonzo Church · Princeton · 1936

Formalized computation using mathematical functions

→ Lambda Calculus

Lambda Calculus $\equiv$ Turing Machine

(same class of computable problems)

A Brief History

John McCarthy · MIT · 1958

Working on LISP — the first high-level language after FORTRAN
Stumbled upon Church's Lambda Calculus paper
→ implemented it in LISP

COBOL didn't exist yet.

Programming with Math

Math and programming

LET $x = x+1$

BASIC somewhere close to 1980

Programming with Math

Math and programming

LET x = x+1

BASIC somewhere close to 1980

$$x = x+1$$

$$\cancel{x} = \cancel{x}+1$$

$$0=1$$

Programming with Math

Math and programming

LET $x = x+1$

~~BASIC somewhere close to 1980~~

$x=3$

is forever

Programming with Math

Math and programming

LET $x = x + 1$

BASIC somewhere close to 1980

$x = 3$

is forever

FP in Three Rules

1

Variables cannot be changed

Once assigned, a value is bound forever.
Want a different value? Make a copy.

FP in Three Rules

1 Variables cannot be changed

2 **Functions are deterministic**

Same input → always same output.

No access to mutable external state.

FP in Three Rules

- 1 Variables cannot be changed
- 2 Functions are deterministic

3 **Functions are first-class citizens**

Assign · Pass · Return · Compose

C++ and Scala

Choosing a touchstone

C++ — Multi-Paradigm?

"If you must use fancy words, C++ is a multi-paradigm programming language."

— Bjarne Stroustrup

C++ — Multi-Paradigm?

"If you must use fancy words, C++ is a multi-paradigm programming language."

— Bjarne Stroustrup

Imperative

C++ — Multi-Paradigm?

"If you must use fancy words, C++ is a multi-paradigm programming language."

— Bjarne Stroustrup

Imperative

OOP

C++ — Multi-Paradigm?

"If you must use fancy words, C++ is a multi-paradigm programming language."

— Bjarne Stroustrup

Imperative

OOP

Generic

C++ — Multi-Paradigm?

"If you must use fancy words, C++ is a multi-paradigm programming language."

— Bjarne Stroustrup

Imperative

OOP

Generic

Logic

C++ — Multi-Paradigm?

"If you must use fancy words, C++ is a multi-paradigm programming language."

— Bjarne Stroustrup

Imperative

OOP

Generic

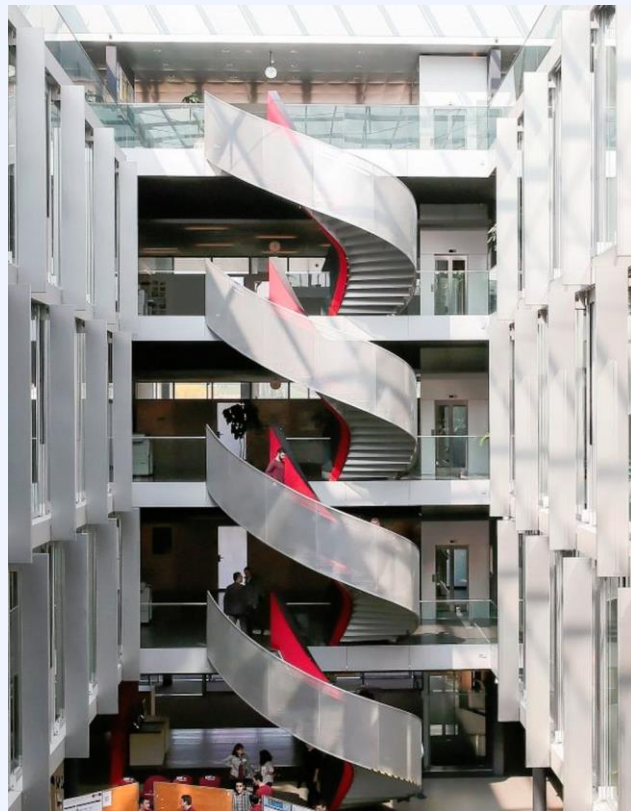
~~Logic~~

Functional

Tonight →

The Touchstone: Scala

- Designed by **Martin Odersky** — EPFL, 2004
- **Multi-paradigm**, runs on the **JVM**
- **First-class FP support** — the features we'll want to compare
- Steep learning curve (we're C++ programmers — we're used to it)
- Pioneered at scale: Twitter, LinkedIn → Zalando, Morgan Stanley, Airbnb



Variables Cannot Be Changed

Pillar 1: Immutability

The Problem with Mutable State

How many times have you scanned code like this?

```
auto x = computeSomething();  
// ... 50 lines ...  
f(x);           // Did f modify x?  
// ... 30 more lines ...  
use(x);         // What is x now?
```

C++

- In FP: once assigned, x never changes
- No defensive scanning. Local reasoning is enough.
- Bonus: no data races in multi-threaded code

Declaring Immutability

C++

```
auto canChange    = 3;  
const auto fixed  = 3;  
// const is opt-in  
// mutable is the default
```

C++

Scala

```
var canChange    = 3  
val fixed        = 3  
// same cost to write  
// immutable is idiomatic
```

SCALA

In C++: const is opt-in. In Scala: val is idiomatic.

Updating a Struct — Immutably

C++

C++

```
struct LogEntry {  
    std::[...]time_point timestamp;  
    std::string component;  
    std::string severity;  
    std::string message;  
};  
  
// entry is a LogEntry  
// want severity = "ERROR"  
LogEntry escalated = {  
    entry.timestamp,  
    entry.component,  
    "ERROR",  
    entry.message  
};
```

Scala

SCALA

```
case class LogEntry(  
    timestamp: Instant,  
    component: String,  
    severity: String,  
    message: String  
)  
  
// entry is a LogEntry  
// want severity = "ERROR"  
val escalated = entry.copy(  
    severity = "ERROR"  
)
```

C++ becomes boilerplate for large structs. Scala's copy() is always terse.

Deterministic Functions

Pillar 2: Pure functions

Pure vs Impure Functions

PURE

- Result depends only on arguments
- No external mutable state
- Trivially unit-testable

Pure Functions can be composed -
New (pure) functions can be created
by assembling basic functions

IMPURE

- Has side effects
- Hard to test without mocking
- Hidden dependencies

I/O,
Random,
Exception
s...

What do you see?

```
template<typename T>  
T max( T a, T b ) { return a > b ? a : b; }
```

C++

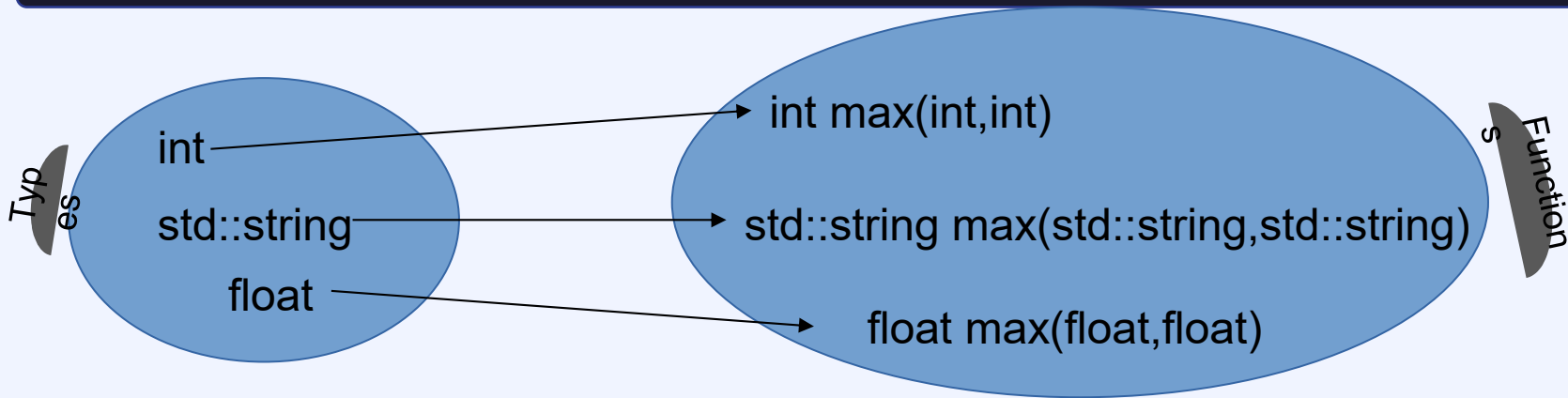
You've Already Written Pure Functions

Template metaprogramming is purely functional.

```
template<typename T>  
T max( T a, T b ) { return a > b ? a : b; }
```

C++

```
// Give it int → get max(int,int)  
// Give it float → get max(float,float)  
// Same input → same output. Always.
```



Metaprogramming is Purely Functional

But also...

- Templates: functions over types
- Template specialisation: pattern matching on types
- constexpr: pure functions over compile-time values
- static_assert: the unit test of pure meta-functions

Quarantine, Don't Eliminate

You don't eliminate side effects — you quarantine them.

Shell: I/O, side effects

Core: pure functions

*Shell calls Core
never the reverse*

parseLine — A Pure Function

C++

```
// Pure: same input → same output, no I/O
// "2026-05-01T10:32:11 [AUTH] ERROR invalid token"
std::optional<LogEntry> parseLine( std::string const& line )
{
    auto parts = split( line, ' ', 4 );
    if( parts.size() < 4 ) return std::nullopt;
    auto timestamp = parseTimestamp( parts[0] ); // timestamp is
std::optional
    if( !timestamp ) return std::nullopt;
    return LogEntry{ *timestamp,
                     trim(parts[1], "["],
                     parts[2],
                     parts[3] };
}
```

parseLine is trivially unit-testable: pass a string, get a result, no mocking.

parseLine — Scala Version

SCALA

```
// Pure: same input → same output, no I/O
def parseLine( line: String ): Option[LogEntry] =
  line.split( " ", 4 ) match
    case Array( ts, component, severity, message ) =>
      val time = parseTimestamp( ts )
      if time.isDefined then
        Some( LogEntry(
          time.get,
          component.stripPrefix("[").stripSuffix("]"),
          severity,
          message
        ))
      else
        None
    case _ => None
```

Pattern matching does the structural parsing. Structurally identical to the C++ version.

We have the inner core.

To build the outer shell and wire the full application together we need more tools

Tools needed first: first-class functions, lambdas, higher-order functions, monads.

Functions are First-Class Citizens

Pillar 3

Function Pointers — Close, But Not Enough

```
typedef B (f_A)( A ); // function: A → B
typedef C (f_B)( B ); // function: B → C

// Can we write compose in C?
typedef C (f_C)( A );

f_C* compose( f_A* f, f_B* g )
{
    // ??? — no place to store f and g
}
```

C++

Function Pointers — Close, But Not Enough

```
typedef B (f_A)( A ); // function: A → B
typedef C (f_B)( B ); // function: B → C

// Can we write compose in C?
typedef C (f_C)( A );

f_C* compose( f_A* f, f_B* g )
{
    // ??? — no place to store f and g
}
```

C++

The problem: a function pointer has **no room to carry captured context**.

compose — Scala vs C++

C++ (template)

C++

```
auto compose( auto f, auto g )
{
    return [f,g]( auto a ){
        return g( f( a ) );
    };
}

// auto = template in disguise
// C++ has no native function type
```

Scala

SCALA

```
def compose(f: A => B, g: B => C) =
    (a: A) => g( f( a ) )

// return type is a function
// not a pointer
```

compose — with std::function

```
std::function<C(A)> compose(  
    std::function<B(A)> f,  
    std::function<C(B)> g )  
{  
    return [f,g]( A a ){  
        return g( f( a ) );  
    };  
}
```

C++

```
// explicit types - but type erasure costs potential allocation
```

Two options in C++: template way (fast, no named type) or std::function (explicit types, possible allocation).

Practical compose — Log Preprocessing

C++

```
auto trim =  
    [](std::string_view s){...};  
auto stripAnsi =  
    [](std::string_view s){...};  
auto normalizeSpaces=  
    [](std::string_view s){...};  
  
auto preProcess = compose(  
    trim,  
    compose(stripAnsi,  
            normalizeSpaces));  
  
auto entry = parseLine(  
    preProcess(rawLine)  
);
```

C++

Scala

SCALA

Practical compose — Log Preprocessing

C++

C++

```
auto trim =  
    [](std::string_view s){...};  
auto stripAnsi =  
    [](std::string_view s){...};  
auto normalizeSpaces=  
    [](std::string_view s){...};  
  
auto preProcess = compose(  
    trim,  
    compose(stripAnsi,  
            normalizeSpaces)  
);  
auto entry = parseLine(  
    preProcess(rawLine)  
);
```

Scala

SCALA

```
val trim = (s:String) => ???  
val stripAnsi = (s:String) => ???  
val normalizeSpaces =  
    (s:String) => ???  
  
val preProcess =  
    trim andThen  
    stripAnsi andThen  
    normalizeSpaces  
  
val entry =  
    parseLine( preProcess(rawLine) )
```

??? is
valid
Scala

andThen is
library for
compose

A Closer Look at Lambdas

Anonymous functions and closures

Lambda Anatomy — C++

```
[ capture-list ]( arguments ) -> return-type { body };
```

C++

- **capture-list:** by value [x], by ref [&x], all by value [=], all by ref [&]
- **arguments:** optional (unlike mathematical functions)
- **return-type:** optional if inferrable from body
- C++ exposes every knob — power at the cost of ceremony

cppreference.com/cpp/language/lambda — prepare to be awed by the full grammar.

Lambda Anatomy — Scala

```
( arguments ) => expression  
argument => expression  
expression with _  
  
( x: Int, y: String ) => x+y.length  
x => 2*x  
_+1
```

SCALA

- **Implicit capture:** values mentioned in expression, are resolved by scope
- **arguments:** can't be optional
- **return-type:** inferred by expression
- **types are deduced:** context provides types.

cppreference.com/cpp/language/lambda — prepare to be awed by the full grammar.

Lambda Verbosity — Side by Side

C++

C++

```
auto isError =  
    []( LogEntry const& e ){  
        return e.severity=="ERROR";  
    };  
  
auto after =  
    []( std::...::time_point t ){  
        return [t](LogEntry const& e){  
            return e.timestamp > t;  
        };  
    };
```

Scala

SCALA

```
val isError: LogEntry => Boolean =  
    _.severity == "ERROR"  
  
val after =  
    (t: Instant) =>  
        (e: LogEntry) =>  
            e.timestamp.isAfter(t)
```

Predicates Are Values

```
auto isError = []( LogEntry const& e ){ return e.severity == "ERROR"; };  
auto isWarning = []( LogEntry const& e ){ return e.severity == "WARNING"; };  
  
// after() returns a predicate – partial application  
auto after = []( std::chrono::system_clock::time_point t ){  
    return [t]( LogEntry const& e ){ return e.timestamp > t; };  
};  
  
// A predicate built from a predicate-factory  
auto isRecent = after(  
    std::chrono::system_clock::now() - std::chrono::hours(1)  
);
```

Store them. Pass them. Compose them. They're just values.

Everything is an Expression

Statements that return values

The Uninitialized Variable Problem

C++ — uninitialized result:

```
OutputFormat format;    // ← no initial value
if( args.contains("--json") )
    format = OutputFormat::Json;
else if( args.contains("--csv") )
    format = OutputFormat::Csv;
else
    format = OutputFormat::PlainText;
```

C++

The Uninitialized Variable Problem

C++ — uninitialized result:

```
OutputFormat format;    // ← no initial value
if( args.contains("--json") )
    format = OutputFormat::Json;
else if( args.contains("--csv") )
    format = OutputFormat::Csv;
else
    format = OutputFormat::PlainText;
```

C++

Scala — if is an expression:

```
val format =
  if args.contains("--json") then OutputFormat.Json
  else if args.contains("--csv") then OutputFormat.Csv
  else OutputFormat.PlainText
```

SCALA

lambda — C++'s Workaround

C++

```
auto const format = [&]{  
    if( args.contains("--json") ) return OutputFormat::Json;  
    if( args.contains("--csv") )  return OutputFormat::Csv;  
    return OutputFormat::PlainText;  
}();  
// format is always initialized  
// branches are self-contained  
// works for switch too  
// BUT...  
// * you need to remember the (); at the end  
// * not very readable (easy to miss the invocation of the lambda)
```

std::invoke — C++'s Workaround

```
auto const format = std::invoke( [&]{  
    if( args.contains("--json") ) return OutputFormat::Json;  
    if( args.contains("--csv") )  return OutputFormat::Csv;  
    return OutputFormat::PlainText;  
});
```

```
// clearer syntax  
// clearer intent  
// no rogue parenthesis to make it work
```

C++

How Do I Solve Problems?

Recursion and higher-order functions

Recursion as Iteration

C++

C++

```
int sum( auto begin, auto end )
{
    if( begin == end )
        return 0;
    return *begin +
        sum( std::next(begin), end );
}
```

Scala

SCALA

```
def sum[T]( list: List[T] ): Int =
    list match
    case Nil =>
        0
    case head :: rest =>
        head + sum( rest )
```

Tail Recursion — Free Loops

Tail-recursive: last operation is the recursive call (no pending work after it).

Compiler rewrites it into a loop → no stack growth.

C++

```
// No guarantee.  
// No annotation.  
// You're on your own.
```

C++

Scala

```
@tailrec  
def sum[T:Numeric](list:List[T]):T=  
  val num = summon[Numeric[T]]  
  list match  
    case Nil => num.zero  
    case head :: rest =>  
      num.plus(head, sum(rest))
```

SCALA

Cannot rewrite recursive call: it is not in tail position

```
@tailrec  
def sum[T: Numeric](list: List[T]): T
```

fo and cplusplus main

Tail Recursion — Free Loops

Tail-recursive: last operation is the recursive call (no pending work after it).

Compiler rewrites it into a loop → no stack growth.

C++

```
// No guarantee.  
// No annotation.  
// You're on your own.
```

C++

Scala

```
@tailrec  
def sum[T:Numeric](  
  list: List[T], acc: T = 0  
): T =  
  val num = summon[Numeric[T]]  
  list match  
    case Nil => acc  
    case head :: tail =>  
      sum( tail, num.plus(acc, head))  
  
// no compiler error b/c it's  
// tail-recursive
```

SCALA

Higher-Order Functions

Abstracting over loops

hasRecentError — Loop Version

```
bool hasRecentError( std::vector<LogEntry> const& entries )
{
    for( auto const& e : entries ){
        if( e.severity == "ERROR" && isRecent(e) )
            return true;
    }
    return false;
}
```

C++

Two concerns mixed together: iteration + condition.

hasRecentError — Ranges Version

```
bool hasRecentError( std::vector<LogEntry> const& entries )
{
    return std::ranges::any_of(
        entries | std::views::filter( isError ),
        isRecent
    );
}
```

C++

- Iteration embedded in ranges — not visible in your code
- Two conditions listed separately — each concern is isolated
- isError and isRecent are the predicates we already built

hasRecentError — Scala Version

```
def hasRecentError( entries: List[LogEntry] ): Boolean =  
  entries  
    .filter( isError )  
    .exists( isRecent )
```

SCALA

Similar structure, no difference between range-views function and

```
collection → collection: filter, map, flatMap  
collection → value:      exists, fold (foldLeft, foldRight)
```

fold — One Value from a Sequence

When you need a single result from a whole collection:

```
collection
  .drop(1)
  .foldLeft( collection.head )(
    (acc, x) => if acc > x then acc else x
  )
```

SCALA

C++20 equivalent: `std::ranges::fold_left` — but NOT usable in pipe syntax.

summarize — fold in Practice

C++

C++

```
Summary
summarize(vector<LogEntry> const& v)
{
    return std::ranges::fold_left(
        v,
        Summary{},
        []( Summary acc,
            LogEntry const& e ){
            if( e.severity=="ERROR" )
                return Summary{
                    acc.errorCount+1, ...
                };
            return acc;
        });
}
```

Scala

SCALA

```
def summarize(
    entries: List[LogEntry]
): Summary =
    entries.foldLeft(
        Summary(0)
    )( (acc, e) =>
        e.severity match
        case "ERROR" =>
            acc.copy(
                errorCount =
                    acc.errorCount + 1
            )
        case _ => acc
    )
```

Transforming Collections

Functor — the FP container interface

What is a Functor?

A container that knows how to transform its contents.

⚠ In FP, 'functor' ≠ callable object (the C++ meaning).
Math was here first with a different definition.

```
// The Functor interface in Scala
trait Functor[F[_]]:
  def map[A, B](fa: F[A])(f: A => B): F[B]

// F[_] = 'F is a type constructor – we don't care what it holds'
// Give it List, Optional, Future... it works the same
```

SCALA

Why C++ Can't Express This

```
// What we'd WANT to write (not valid C++):  
template<typename A>  
concept Functor = requires {  
    template<typename B>  
    Functor<B> map( std::function<B(A)>&& f ) const;  
};  
// Problem: concepts can't be parametrized over type constructors  
// This is the 'higher-kinded types' gap
```

C++

- C++ concepts can constrain a type — not a type constructor
- No WG21 proposal for higher-kinded types is currently active
- `std::transform` exists as a free function — but no unified Functor interface

optional as a Functor — Before C++23

Ignoring optional's functor nature:

C++

```
std::optional<std::string> extractComponent( std::string const& line )
{
    std::optional<LogEntry> entry = parseLine( line );
    if( !entry.has_value() ) return std::nullopt; // manual guard
    return entry->component;                     // manual extract
}
```

Every optional-returning function repeats this `has_value()` ceremony.

optional as a Functor — After

C++23

C++

```
std::optional<std::string>
extractComponent(
    std::string const& line
)
{
    return parseLine( line )
        .transform([](LogEntry const& e){
            return e.component;
        });
}

// transform = map for optional
// (P0798, C++23)
```

Scala

SCALA

```
def extractComponent(
    line: String
) : Option[String] =
    parseLine( line ).map(_.component)
```

The empty case propagates automatically. You write only the happy path.

Skip the Applicative

Applicative — running parallel computation

Interesting, but not mandatory for introductory FP

Transforming and Flattening

Monad — flatMap and the nesting problem

The Nesting Problem

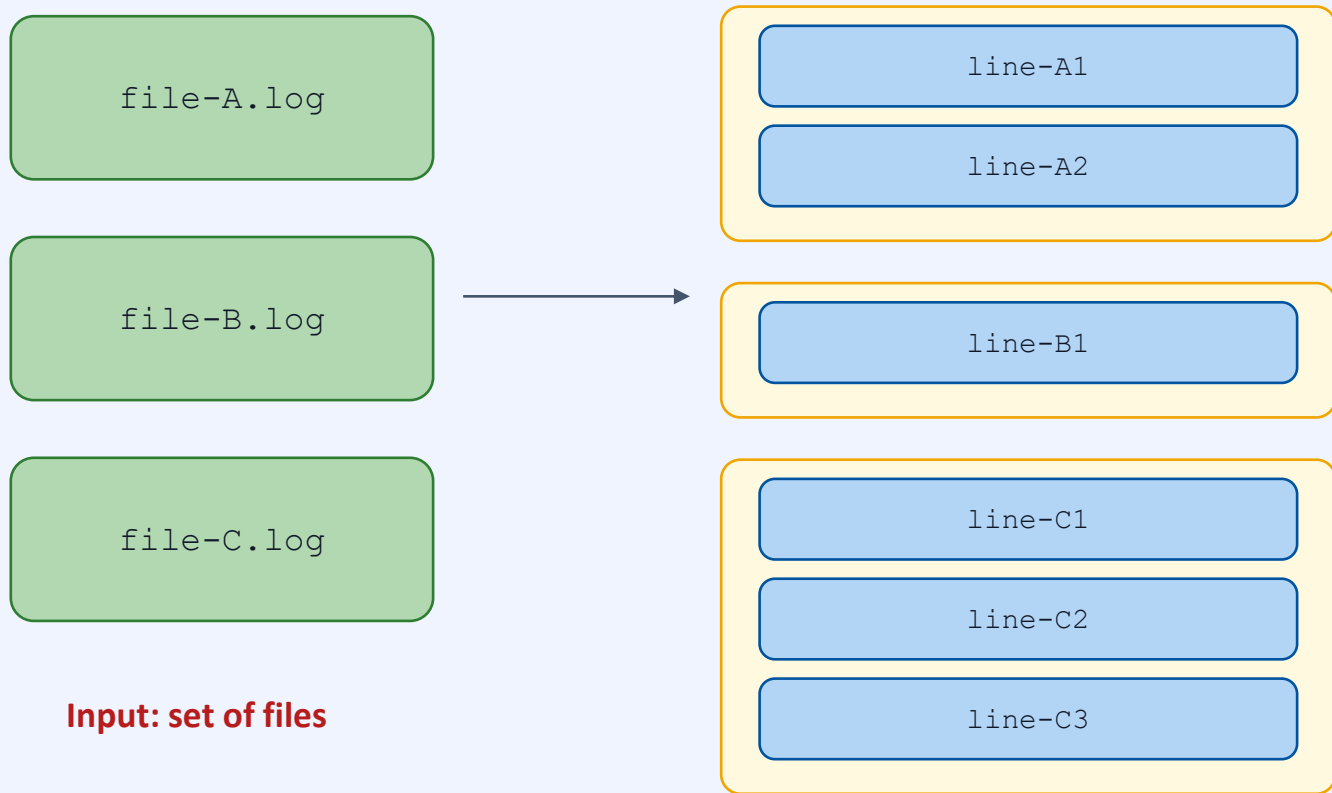
Suppose we have a second step that can also fail:

```
std::optional<std::string> lookupFullName( std::string const& shortName );  
  
// What if we chain it with transform?  
parseLine(line).transform(  
    [] ( LogEntry const& e ) { return lookupFullName( e.component ); }  
);  
// Result type: optional< optional<string> > ← not what we want
```

C++

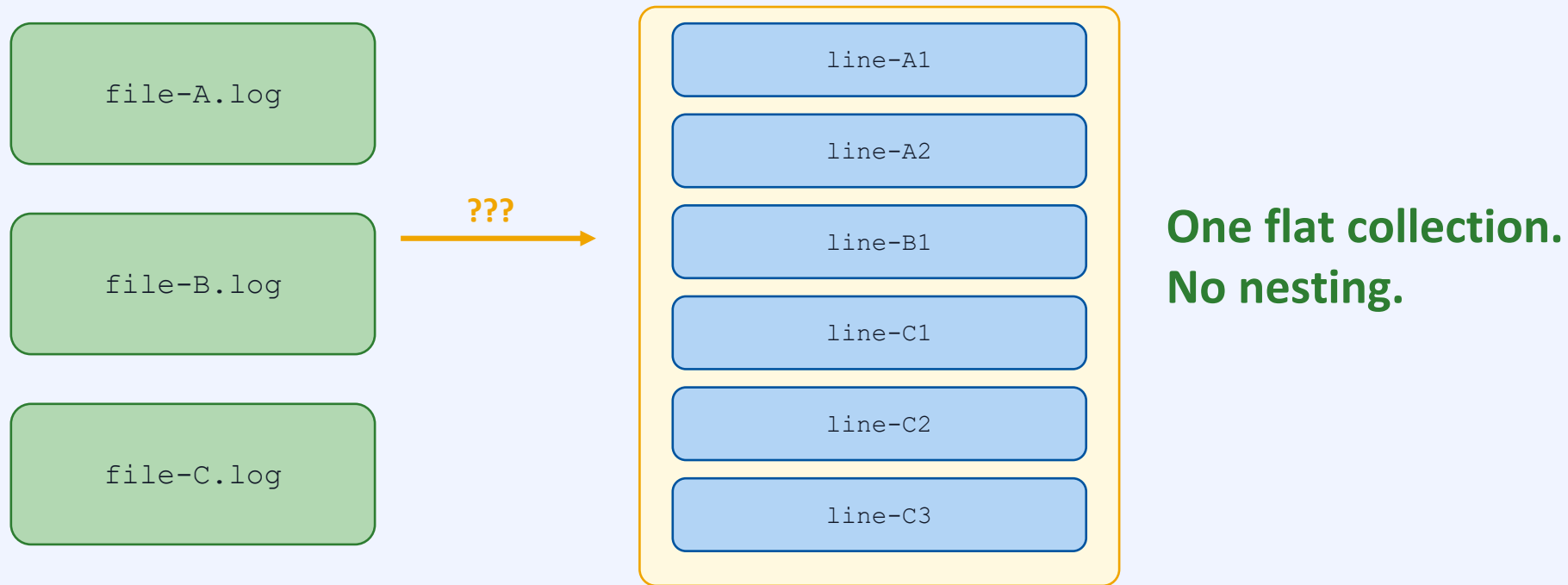
transform maps $A \rightarrow B$. If B is already `optional<...>` we get double-wrapping.

map on a Set of Files



**Result: a set of sets of lines.
Not what we want.**

What we would like



The Monad Interface

*The curse of the monad: once you understand it, you can't explain it.
So I won't try. Here's the interface:*

pure

$A \rightarrow \text{Monad}\langle A \rangle$

Wrap a plain value into the monad

flatMap

$\text{Monad}\langle A \rangle, (A \rightarrow \text{Monad}\langle B \rangle) \rightarrow \text{Monad}\langle B \rangle$

Chain without double-wrapping

Functor transforms a container. Monad avoids nesting when operations are chained.

flatMap in Action — and_then

C++23

```
optional<string> lookupFullName(C++
string const& name );
optional<string>
extractFullComponent(
    string const& line )
{
    return parseLine( line )
        .transform([](LogEntry const& e){
            return e.component;
        })
        .and_then( lookupFullName );
}
// and_then = flatMap for optional
```

Scala

```
def lookupFullName(name: String): SCALA
    Option[String] = ???

def extractFullComponent(
    line: String ) : Option[String] =

    parseLine( line )
        .map( _.component )
        .flatMap( lookupFullName )

// chain is flat — no nesting
// None at any step → done
```

and_then is C++ for flatMap, as transform is C++ for map.

One Abstraction, Two Problems

optional chaining

Container: 0 or 1 item

FlatMap: short-circuit on failure

List / vector flatMap

Container: many items

FlatMap: expand and flatten

Same interface — pure + flatMap — different container semantics.

The C++ Monad Gap

- **std::optional** and **std::expected** — **monadic ops added (C++23)**
- **std::vector**, **std::map** — **no flatMap** (`views::join` + `transform` gets you there, but no standard name)
- **std::future** — **no monadic ops**; proposals (N3857+) never landed; coroutines became the committee's answer
- **Higher-kinded types: no WG21 proposal — impossible to express the Monad interface as a concept**
- **libfn/functional** — experimental library targeting this gap, building on C++26 features

For Comprehension

Syntactic sugar over flatMap

collectErrors — C++ Nested Loops

```
std::vector<std::string> readLines( std::filesystem::path const& path );  
  
std::vector<LogEntry>  
collectErrors( std::vector<std::filesystem::path> const& files )  
{  
    std::vector<LogEntry> result;  
    for( auto const& path : files )  
        for( auto const& line : readLines(path) ){  
            auto e = parseLine( line );  
            if( e && isError(*e) )  
                result.push_back( *e );  
        }  
    return result;  
}
```

result is mutated every loop. Neither std::vector nor any container has flatMap.

collectErrors — For Comprehension

```
def readLines(path: Path) : List[String] = ???
```

SCALA

```
def collectErrors( files: List[Path] ): List[LogEntry] =  
  for  
    path  <- files                // iterate files  
    line  <- readLines( path )    // iterate lines per file  
    entry <- parseLine( line )    // Option — skips None  
    if isError( entry )          // guard  
  yield entry
```

- Iteration part separated from result-making part (yield)
- parseLine returns Option — None cases silently skipped
- This is just flatMap all the way down — desugaring on next slide

For Comprehension is flatMap

```
// The for comprehension "desugars" to:  
files  
  .flatMap(path =>  
    readLines(path)  
      .flatMap(line =>  
        parseLine(line)  
          .withFilter(entry =>  
            isError(entry)  
          )  
          .map(entry =>  
            entry  
          )  
        )  
      )  
  )
```

SCALA

Same operations. For comprehension is syntactic sugar — and it's worth every calorie.

For Comprehension over Future

Each step is async and can fail. For comprehension sequences them without blocking:

```
def enrichAndReport( line: String ): Future[Report] =  
  for  
    entry    <- optionToFuture(parseLine(line))  
    service  <- registry.lookupService( entry.component )  
    ticket   <- ticketSystem.findExisting( entry.message )  
  yield  
    Report( entry, service, ticket )  
// Each <- is a flatMap  
// If any step fails → chain short-circuits, no callbacks needed
```

SCALA

Without For Comprehension — C++

```
std::future<Report> enrichAndReport( std::string const& line )
{
    return optionalToFuture( parseLine(line) )
        .and_then( [&]( LogEntry const& entry ){
            return registry.lookupService( entry.component )
                .and_then( [&]( Service const& service ){
                    return ticketSystem.findExisting( entry.message )
                        .and_then( [&]( std::optional<Ticket> ticket ){
                            return std::experimental::make_ready_future(
                                Report{ entry, service, ticket }
                            );
                        });
                });
        });
}
```

C++

*Same semantics. The nesting accumulates visually. This is why for comprehension exists. Anyway std::future has **no flatMap** (or **and_then**), and there's **no make_ready_future***

Back to I/O

Completing the shell/core story

Wiring the Application — Shell

C++

```
// Shell: reads from filesystem
std::vector<std::string> readLines( std::filesystem::path const& p )
{
    std::vector<std::string> lines;
    std::ifstream file(p);
    for( std::string line; getline(file,line); )
        lines.push_back( std::move(line) );
    return lines;
}

// Shell: writes results
void printSummary( Summary const& s )
{
    std::println( std::cout, "Errors:   {}", s.errorCount );
    std::println( std::cout, "Warnings: {}", s.warningCount );
}
```

Shell functions: I/O only. No logic.

Wiring the Application — main

C++

```
int main( int argc, char* argv[] )
{
    auto lines    = readLines( argv[1] ); // Shell I/O
    auto entries  = parseLines( lines );  // Core (pure)
    auto summary  = summarize( entries ); // Core (pure)
    printSummary( summary );              // Shell I/O
}
```

Shell → Core → Core → Shell. The pure functions are trivially unit-testable.

Wiring the Application — Scala

SCALA

```
// Wiring
@main def run( path: String ): Unit =
  val lines = readLines( Path.of(path) ) // Shell I/O
  val entries = parseLines( lines )      // Core (pure)
  val summary = summarize( entries )     // Core (pure)
  printSummary( summary )                // Shell I/O
```

Structurally identical to C++. readLines and printSummary have side effects — and only they do.

Making the Contract Enforceable

- **C++ and Scala:** quarantine is a **design convention** — not enforced by the type system
- **Haskell:** purity is part of the **function signature** — sneaking in a `printf` is a compile error
- Scala with IO monad: **describes I/O without executing it** — the description is pure

$\text{IO}[A]$ = a description of a side-effecting computation
that will eventually produce an A — but hasn't run yet.

Constructing $\text{IO}[A]$ is pure. Executing it happens exactly once, at the top of `main`.

IO Applied to the Log Tool — Scala

SCALA

```
val program: IO[Unit] =  
  for  
    lines    <- IO( readLines( Path.of("app.log") ))  
    entries  = parseLines( lines )           // pure – no IO wrapper needed  
    summary  = summarize( entries )         // pure  
    _        <- IO( printSummary( summary ) )  
  yield ()  
  
// One place, one moment, the whole world changes:  
program.unsafeRunSync()
```

- program is a **pure value** — nothing has executed yet
- unsafeRunSync() is the only call site of any side effect

IO in C++ — If We Had the Type

```
IO<void> program()
{
    return IO( []{ return readLines( "app.log" ); })
        .andThen( []( std::vector<std::string> const& lines ){
            return IO<std::vector<LogEntry>>::pure( parseLines(lines) );
        })
        .andThen( []( std::vector<LogEntry> const& entries ){
            return IO<Summary>::pure( summarize(entries) );
        })
        .andThen( []( Summary s ){
            return IO<void>( [s]{ printSummary(s); });
        });
}

int main() { program().run(); }
```

C++

No IO<T> in the standard. std::future is the closest spirit — a deferred computation. libunifex / C++ executors go further.

Conclusions

What to take home

Where C++ Falls Short

Language gaps

- No FP syntactic sugar for lambdas (very verbose)
- No for-comprehension equivalent
- No higher-kinded types → can't express Functor/Monad as concepts

Library gaps

- `std::optional` / expected: monadic ops added (C++23) ✓
- `std::future`: proposals failed, coroutines instead
- `std::vector`, `std::map`: no `flatMap`
- No standard IO monad

What FP Principles Give You



Local reasoning

A pure function is fully described by its signature. No hidden surprises.



Non-mutability

Immutable values eliminate a class of bugs and data races.



Abstraction over loops

filter, map, fold, flatMap replace error-prone index management.



Testability

Push I/O to the edges — the core is trivially unit-testable.

Three things to take home

1

Use const pervasively

You'll reduce the number of surprises.

Three things to take home

1 Use const pervasively

2 Use ranges instead of loops

You'll avoid those pesky off-by-one errors.

Three things to take home

- 1 Use const pervasively
- 2 Use ranges instead of loops

3 **Push I/O to the periphery**

Your code's testability will improve.

Resources

Talks

- [Gašper Ažman - "\(Fun\)ctional C++ and the M-word" - C++ London](#)
- [Bartosz Milewski "Monads for C++" - itCppCon17](#)
- [David Sankel - "Monoids, Monads, and Applicative Functors: Repeated Software Pattern" - CppCon 2020](#)

Libraries

- [libfn/functional](#) small and experimental library with the ambitious goal to build a monadic vocabulary on top of `std::optional` and `std::expected`, explicitly hoping pieces of it get folded into a future standard.
- [Functional Plus](#) header only library supporting functional programming
- [chef-fun](#) my attempt at implementing functional Option and Either (the Scala equivalent to `std::expected`).

Standard proposals: P0798 (optional monadic ops) · N3857 (future .then — not adopted)

Resources

Curious about Scala?

- <https://docs.scala-lang.org/>
- <https://deepwiki.com/scala/docs.scala-lang/1.2-learning-resources>

Functional Programming in C++

from a Scala perspective

Questions?